

Продукты



Решения



Разработчики



Партнеры



Цены

Вход

Регистрация

Обучающие материалы

Вопросы

Документация по продукту

Поиск

Обучающие материалы / Ansible / Как установить и настроить Ansible в Rocky Linux 9

Учебник

# Как установить и настроить Ansible в Rocky Linux 9

Обновлено 22 июня 2026 года



Ансибль

Управление конфигурацией

Rocky Linux 9



Авторы: [Брайан Хоган](#), [Джанель Хоркаситас](#) и [Маникандан Куруп](#)

## Не используете Rocky Linux 9?

Выберите другую версию или дистрибутив.

Версия

Rocky Linux 9



Содержание



Популярные темы



## Введение



Системы управления конфигурацией предназначены для того, чтобы администраторы и операционные команды могли контролировать большое количество серверов. Они позволяют автоматизированно управлять множеством различных систем из одного центрального места. Несмотря на то, что существует множество популярных систем управления конфигурацией для [Linux](#)-систем, таких как [Chef](#) и [Puppet](#), они зачастую сложнее, чем нужно многим пользователям. [Ansible](#) — отличная альтернатива этим вариантам, поскольку для начала работы с ней требуется гораздо меньше усилий.

Ansible настраивает клиентские машины с компьютера, на котором установлены и настроены компоненты Ansible. Для получения информации с удаленных машин, выполнения команд и копирования файлов используется обычный канал SSH. Благодаря этому система Ansible не требует установки дополнительного программного обеспечения на клиентских компьютерах. Это один из способов, с

помощью которых Ansible упрощает администрирование серверов: любой сервер с открытым портом SSH может быть настроен с помощью Ansible, независимо от того, на каком этапе жизненного цикла он находится.

В Ansible используется модульный подход, позволяющий использовать функциональность основной системы для решения конкретных задач. Модули могут быть написаны на любом языке и взаимодействовать друг с другом через стандартный JSON. Файлы конфигурации в основном пишутся в формате сериализации данных YAML из-за его выразительности и сходства с популярными языками разметки. Ansible может взаимодействовать с клиентами либо через инструменты командной строки, либо через конфигурационные скрипты, называемые плейбуками.

**Rocky Linux 9** — это бесплатный дистрибутив, поддерживаемый сообществом и совместимый на двоичном уровне с **Red Hat Enterprise Linux (RHEL) 9**, что делает его популярным выбором для *контрольного узла* Ansible (компьютера, на котором запущен Ansible) и *управляемых узлов* (серверов, которые настраивает Ansible). В этом руководстве вы установите Ansible на сервер Rocky Linux 9, настроите аутентификацию с помощью **SSH-ключа**, создадите список хостов и выполните свои первые разовые команды и базовый плейбук. В итоге у вас будет рабочий узел управления, который сможет подключаться к одному или нескольким удаленным серверам Rocky Linux 9 и управлять ими.

### Ключевые рекомендации на вынос

- В Rocky Linux 9 Ansible устанавливается как пакет `ansible-core` из репозитория AppStream, который включен по умолчанию. Более старый универсальный пакет `ansible` больше не доступен в AppStream.
- В Rocky Linux 9 в AppStream установлена версия `ansible-core 2.14.x`, и Red Hat будет поддерживать эту версию до конца жизненного цикла RHEL 9, поэтому она не обновляется с каждым минорным релизом.
- Чтобы получить более новую версию движка или полный пакет сообщества, установите пакет `ansible` из EPEL или установите `ansible / ansible-core` с помощью `pip` вместо AppStream.
- Для управляющего узла требуется Python 3 (Python 3.9 или новее для `ansible-core 2.14`); для управляемых узлов нужен только Python 3 и доступ по SSH, а не сам Ansible.
- Ansible подключается к управляемым узлам по SSH, поэтому аутентификация по ключу и доступность SSH-порта (по умолчанию 22) являются основой для корректной работы.

- Хосты отслеживаются в файле инвентаризации, по умолчанию `/etc/ansible/hosts` , но рекомендуется использовать локальную инвентаризацию проекта, передаваемую с помощью флага `-i` .
- В современных инвентаризациях используются переменные подключения `ansible_host` и `ansible_user` . Старые имена `ansible_ssh_host` и `ansible_ssh_user` по-прежнему работают как псевдонимы, но являются устаревшими.
- Модуль `ping` ( `ansible all -m ping` ) — это стандартный способ убедиться, что управляющий узел может связаться с каждым управляемым узлом, прежде чем запускать реальные задачи.
- AlmaLinux 9 and CentOS Stream 9 use the same AppStream and EPEL repositories, so the installation commands in this guide apply to them as well.

---

## Prerequisites



To follow this tutorial, you will need:

- **One Ansible control node:** The Ansible control node is the machine you will use to connect to and control the Ansible hosts over SSH. Your control node can either be your local machine or a server dedicated to running Ansible, though this guide assumes your control node is a Rocky Linux 9 system. Make sure the control node has:
  - A non-**root** user with `sudo` privileges. To set this up, you can follow **Steps 2 and 3** of our [Initial Server Setup with Rocky Linux 9](#) guide. If you are using a remote server as your control node, you should follow **every step** of that guide.
  - An SSH keypair associated with this user. To set this up, you can follow **Step 1** of our guide on [How to Set Up SSH Keys on Rocky Linux 9](#).
  - Python 3 installed. Rocky Linux 9 includes Python 3.9 by default, which satisfies the requirement of `ansible-core` 2.14 for a control node.
- **One or more Ansible hosts (managed nodes):** An Ansible host is any machine that your control node is configured to automate. This guide assumes your Ansible hosts are remote Rocky Linux 9 servers. Each managed node only needs Python 3 and SSH access; Ansible itself does not need to be installed on it. Make sure each host has:

- The control node's SSH public key added to the `authorized_keys` of a system user. This user can be either **root** or a regular user with `sudo` privileges. To set this up, you can follow **Step 2** of [How to Set Up SSH Keys on Rocky Linux 9](#).

Once you are done setting everything up, you are ready to begin the first step.

---

## Step 1: Installing Ansible

To begin using Ansible to manage your various servers, you first need to install the Ansible software on the machine that will serve as your control node.

On Rocky Linux 9, the Ansible engine is packaged as `ansible-core` and lives in the **AppStream** repository, which is enabled by default. This is a change from Rocky Linux 8, where the all-in-one `ansible` package was installed from EPEL. On Rocky Linux 9, the `ansible` package is no longer provided by [AppStream](#), so you install the engine directly with `dnf`.

Before installing, you can confirm which version AppStream offers by querying the package:

Copy

```
$ sudo dnf info ansible-core
```

The output lists the candidate version and the repository it comes from, similar to the following:

Output

Copy

```
Available Packages
Name           : ansible-core
Epoch         : 1
Version        : 2.14.18
Release        : 3.el9
Architecture   : x86_64
Size           : 2.2 M
Source         : ansible-core-2.14.18-3.el9.src.rpm
Repository     : appstream
Summary        : SSH-based configuration management, deployment, and task execu
```

```
URL      : http://ansible.com
License  : GPLv3+
Description : Ansible is a radically simple model-driven configuration management
            : multi-node deployment, and remote task execution system. Ansible
            : over SSH and does not require any software or daemons to be installed
            : on remote nodes. Extension modules can be written in any language
            : and are transferred to managed machines automatically.
```

With the version confirmed, install `ansible-core` :

Copy

```
$ sudo dnf install ansible-core
```

Rocky Linux 9 ships `ansible-core 2.14.x`, and Red Hat holds this version for the remainder of the RHEL 9 lifecycle rather than advancing it with each minor release. This means the AppStream version is stable and well-tested but will not be the newest release available upstream. If you need a newer engine or the full community collection bundle, see the alternatives below.

## Installing a newer version from EPEL or pip

The default AppStream package installs only the core engine and a small set of built-in modules. If you want the larger `ansible` community package, which bundles `ansible-core` together with a curated set of community collections, you have two main options.

The first option is the **EPEL** (Extra Packages for Enterprise Linux) repository. Enable it and install the bundled package:

Copy

```
$ sudo dnf install epel-release
$ sudo dnf install ansible
```

Because the EPEL `ansible` package depends on a compatible `ansible-core`, version mismatches between AppStream and EPEL can occasionally cause dependency errors. If `dnf` reports a conflict, prefer a single source rather than mixing the two. Package availability and versions in EPEL can also change over time, so you can confirm what is currently offered with `sudo dnf search ansible` before installing.

The second option is **pip**, the Python package manager, which always offers the most recent release. Installing into a virtual environment keeps it isolated from the system packages:

Copy

```
$ python3 -m venv ~/ansible-venv
$ source ~/ansible-venv/bin/activate
$ pip install ansible
```

The standalone `ansible` package on PyPI tracks the newest community release and pulls in a recent `ansible-core` , so pip is the best route when you specifically need the latest features. Because these versions advance regularly, check the [Ansible release status](#) for the current pairing rather than relying on a fixed version number. Keep in mind that `ansible-core` 2.18 and later require [Python](#) 3.11 or newer on the control node, so confirm your Python version before upgrading this way.

To compare the three installation sources at a glance, refer to the following table:

| Source              | Package                 | Typical Version on Rocky Linux 9   | Includes Community Collections   | Best For                           |
|---------------------|-------------------------|------------------------------------|----------------------------------|------------------------------------|
| AppStream (default) | ansible-core            | 2.14.x (held for RHEL 9 lifecycle) | No                               | Stable, supported, no extra repos  |
| EPEL                | ansible                 | Tracks a recent community release  | Yes                              | Bundled collections from a repo    |
| pip / PyPI          | ansible or ansible-core | Latest upstream release            | ansible : yes; ansible-core : no | Newest features, isolated installs |

## Verifying the installation

After installing, confirm that Ansible is available and check which version and Python interpreter it is using:

Copy

```
$ ansible --version
```

The output reports the version, configuration file paths, and the Python executable Ansible runs under:

Output

Copy

```
ansible [core 2.14.18]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/sammy/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.9/site-packages/ansible
  ansible collection location = /home/sammy/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.9.16 (main, Dec  8 2022, 00:00:00) [GCC 11.3.1 20221121]
  jinja version = 3.1.2
  libyaml = True
```

Now you have all of the software required to administer your servers through Ansible.

## Installing collections when you need them

The `ansible-core` package ships only the built-in `ansible.builtin` modules. If a playbook needs a module from a community collection, for example, something under `community.general`, `ansible-core` will report that the module cannot be found. In that case, install the collection separately with `ansible-galaxy`:

Copy

```
$ ansible-galaxy collection install community.general
```

You only need this step when a task references a module outside `ansible.builtin`. If you installed the full `ansible` package from EPEL or pip instead of `ansible-core`, the most common collections are already bundled and this step is usually unnecessary.

---

## Step 2: Configuring Ansible hosts ^

Ansible keeps track of all of the servers it knows about through an *inventory* file. The default location is `/etc/ansible/hosts`, and you need to set up an inventory before you can communicate with your other machines.

The `ansible-core` package does not always create the `/etc/ansible` directory or a sample `hosts` file, so create the directory first if it does not exist:

Copy

```
$ sudo mkdir -p /etc/ansible
```

With the directory in place, open the inventory file with **root** privileges. The default text editor on Rocky Linux 9 is `vi`, so this example uses it:

Copy

```
$ sudo vi /etc/ansible/hosts
```

The inventory file is flexible and can be configured in a few different ways. The simplest syntax uses an INI-style format that looks like the following:

Example inventory file

Copy

```
[ group_name ]  
alias ansible_host= your_server_ip
```

The `group_name` is an organizational tag that lets you refer to any servers listed under it with one word, and the alias is a friendly name you use to refer to an individual server. The `ansible_host` variable tells Ansible the address to connect to.

For example, imagine you have three servers you want to control with Ansible. Because Ansible communicates with managed nodes over SSH, each server must be reachable

from the control node. If you followed the **One or more Ansible hosts** option in the prerequisites, your hosts already have the control node's SSH key installed, so you can connect without a password:

Copy

```
$ ssh sammy @ your_server_ip
```

You will not be prompted for a password. While Ansible can handle password-based [SSH authentication](#), SSH keys keep things more streamlined and secure.

With your inventory file open, add a block that uses the example IP addresses `203.0.113.111` , `203.0.113.112` , and `203.0.113.113` . Make sure to replace these with your own server addresses. This setup lets you refer to each server individually as `host1` , `host2` , and `host3` , or to all of them at once as the `servers` group. To enter this in `vi` , press `i` to switch to insert mode, type the block, then press `ESC` :

`/etc/ansible/hosts`

Copy

```
[servers]
host1 ansible_host= 203.0.113.111
host2 ansible_host= 203.0.113.112
host3 ansible_host= 203.0.113.113
```

Once you are done adding the block, save and exit the file by typing `:wq` and then `ENTER` .

Note that this guide uses `ansible_host` , the modern connection variable. Older tutorials and inventories often use `ansible_ssh_host` ; that name still works as a legacy alias, but `ansible_host` is the current recommended form. The same applies to `ansible_user` , which replaced the older `ansible_ssh_user` .

Hosts can belong to multiple groups, and groups can set parameters for all of their members. You can test the connection to a single host with the following command. Note that the host pattern comes first, immediately after `ansible` , which is the conventional ordering shown in the official Ansible documentation:

Copy

```
$ ansible host1 -m ping
```

If Ansible cannot connect with the current settings, it returns an error like the following:

Ansible connection error

Copy

```
host1 | UNREACHABLE! => {
  "changed": false,
  "msg": "Failed to connect to the host via ssh: sammy@203.0.113.111: Permi
  "unreachable": true
}
```

By default, Ansible tries to connect to remote hosts using your current local username. This error shows that if that user does not exist on the remote system, the connection fails.

To fix this, tell Ansible which remote user to use for the `servers` group. Begin by creating a `group_vars` directory in the Ansible configuration structure:

Copy

```
$ sudo mkdir -p /etc/ansible/group_vars
```

Within this folder, you create one file per group you want to configure. The file is named after the inventory group it applies to, so the file for the `servers` group is simply named `servers`. Ansible loads these files based on the name, so a plain `servers` file works; a `.yaml` extension ( `servers.yaml` ) is also commonly used and is equally valid. You can edit the file with any text editor. The examples below use `nano` ; if it is not already installed and you prefer it over `vi` , install it first:

Copy

```
$ sudo dnf -y install nano
```

Open the group variables file for the `servers` group:

Copy

```
$ sudo nano /etc/ansible/group_vars/servers
```

Add the following to the file. YAML files start with `---` , so be sure to include that line:

/etc/ansible/group\_vars/servers

Copy

```
---
ansible_user: sammy
```

Once you are finished, save and exit the file. In `nano` , press `CTRL + X` , then `Y` , then `ENTER` .

Now Ansible will always use the `sammy` user for the `servers` group, regardless of which user is currently logged in. If you want to set configuration details for every server regardless of group, place them in a file at `/etc/ansible/group_vars/all` . You can configure individual hosts by creating files under a directory at `/etc/ansible/host_vars` .

If you would rather keep your inventory inside a project folder instead of the system-wide location, save it as a file such as `inventory.ini` and pass it to any command with the `-i` flag, for example `ansible all -i inventory.ini -m ping` . A project-local inventory is the recommended modern practice because it keeps each project self-contained and version-controllable.

---

## Step 3: Using basic Ansible commands

^

Now that your hosts are set up with enough configuration to connect successfully, you can try out several ad-hoc commands. Ad-hoc commands run a single module against your hosts without writing a playbook, which is ideal for quick tasks and testing.

First, ping all of the servers you configured. The `-m ping` portion of the command tells Ansible to use the `ping` module. This module operates somewhat like the normal `ping` utility in Linux, but instead of sending **ICMP** packets it checks for Ansible connectivity over SSH and confirms a usable Python interpreter on the remote host:

Copy

```
$ ansible all -m ping
```

Ansible returns output like the following, with one block per host:

Output

Copy

```
host3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
host1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
host2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

A **pong** reply from every host confirms that Ansible has a working connection to all of its managed nodes.

Beyond targeting **all**, you can scope a command to different sets of servers. To target a group, name it directly:

Copy

```
$ ansible servers -m ping
```

To target an individual host, use its alias:

Copy

```
$ ansible host1 -m ping
```

To target several specific hosts, separate them with a comma. A comma is the preferred separator in current Ansible versions, though a colon also works:

Copy

```
$ ansible 'host1,host2' -m ping
```

The `shell` module lets you send a terminal command to the remote host and retrieve the result. For instance, to check memory usage on `host1`, run the following:

Copy

```
$ ansible host1 -m shell -a 'free -m'
```

You pass arguments to a module with the `-a` switch. The command returns output similar to this:

Output

Copy

```
host1 | CHANGED | rc=0 >>
      total        used          free      shared  buff/cache   available
Mem:    7951        234        6768           0         948           7
Swap:    0           0           0
```

Ad-hoc commands are also handy for system management. For example, you can check uptime across every host, or use the `dnf` module to install a package on all servers in the `servers` group. Because installing a package requires elevated privileges, add the `--become` flag (often abbreviated `-b`) so Ansible escalates to **root** with `sudo`:

Copy

```
$ ansible servers -b -m dnf -a "name=htop state=present"
```

You have now run several of the basic Ansible commands across your various hosts.

---

## Step 4: Running a basic playbook

Ad-hoc commands are useful for one-off tasks, but the real strength of Ansible is the *playbook*, a YAML file that describes a desired state and can be run repeatedly with the same result. The following example installs and starts the `chrony` time synchronization service on every host in the `servers` group.

Create a file named `playbook.yml` in your home directory:

Copy

```
$ nano playbook.yml
```

Add the following content, which defines one play with two tasks:

playbook.yml

Copy

```
---
- name: Basic server setup
  hosts: servers
  become: true
  tasks:
    - name: Ensure chrony is installed
      ansible.builtin.dnf:
        name: chrony
        state: present

    - name: Ensure chrony is running and enabled
      ansible.builtin.service:
        name: chronyd
        state: started
        enabled: true
```

Save and close the file, then run it with the `ansible-playbook` command:

Copy

```
$ ansible-playbook playbook.yml
```

Ansible reports the status of each task per host and finishes with a recap:

Output

Copy

```
PLAY [Basic server setup] *****

TASK [Gathering Facts] *****
ok: [host1]
ok: [host2]
ok: [host3]

TASK [Ensure chrony is installed] *****
changed: [host1]
changed: [host2]
changed: [host3]

TASK [Ensure chrony is running and enabled] *****
changed: [host1]
changed: [host2]
changed: [host3]

PLAY RECAP *****
host1  : ok=3    changed=2    unreachable=0    failed=0    skipped=0
host2  : ok=3    changed=2    unreachable=0    failed=0    skipped=0
host3  : ok=3    changed=2    unreachable=0    failed=0    skipped=0
```

Because playbooks are idempotent, running the same playbook a second time reports `changed=0` , since the desired state already matches reality. To go deeper into playbook syntax and structure, see our guide on [Configuration Management 101: Writing Ansible Playbooks](#).

---

## Troubleshooting common connection issues



Rocky Linux 9 ships with both `firewalld` and `SELinux` enabled by default. Neither normally blocks a standard Ansible connection, but a few situations can interrupt connectivity, so it helps to know where to look. Alongside the firewall and SELinux, two SSH- and Python-related errors are among the first that new users hit. The following subsections cover the most common blockers.

## Firewalld blocks the SSH port

Ansible reaches managed nodes over SSH, so the SSH port must be open in the firewall. By default, `firewalld` permits the `ssh` service on port 22, so a default install works without changes. If you moved SSH to a non-standard port or removed the default rule, connections will time out. On the managed node, confirm that SSH is allowed and add it if necessary:

Copy

```
$ sudo firewall-cmd --permanent --add-service=ssh
$ sudo firewall-cmd --reload
```

If you run SSH on a custom port, open that specific port instead, for example `sudo firewall-cmd --permanent --add-port=2222/tcp`.

## SELinux interferes with a module's file operations

SELinux in enforcing mode does not block ordinary SSH logins, so basic `ping` connectivity usually succeeds even with SELinux on. Problems tend to appear when a module writes to a location with an unexpected security context. To check whether SELinux is enforcing, run the following on the managed node:

Copy

```
$ getenforce
```

If the output is `Enforcing` and a task fails with a permission error that ordinary file permissions do not explain, inspect the audit log for denials:

Copy

```
$ sudo ausearch -m avc -ts recent
```

The reported denial usually names the context that needs adjusting. Set the correct context with `restorecon` or `semanage fcontext` rather than disabling SELinux, which keeps the security benefits in place. For managing SELinux states and booleans from Ansible itself, install the `python3-libselinux` package on the managed nodes so the relevant modules can run.

A common real-world case occurs when a playbook uses the `copy` or `template` module to write a file into a non-standard location, such as a custom web root outside `/var/www`. The file copies successfully, but the web server cannot serve it because the new file does not have the `httpd_sys_content_t` context SELinux expects. The fix is to relabel the target directory rather than turn SELinux off. You can do this from a task in your playbook with the `ansible.builtin.sefcontext` and `ansible.builtin.command` modules (running `restorecon`), or manually on the managed node:

Copy

```
$ sudo semanage fcontext -a -t httpd_sys_content_t "/opt/mysite(/.*)?"
$ sudo restorecon -Rv /opt/mysite
```

## Ansible cannot find a Python interpreter

Because Ansible runs Python code on the managed node, it needs a usable Python 3 interpreter there. On minimal server images, Python may be missing or installed at a path Ansible does not expect, and a task fails with an error such as:

Output

Copy

```
Failed to find a Python interpreter
```

Rocky Linux 9 includes Python 3 by default, but if a managed node lacks it, install it:

Copy

```
$ sudo dnf install python3
```

If Python is present but at a non-standard path, tell Ansible where to find it by setting the `ansible_python_interpreter` variable for the host or group. For example, in `/etc/ansible/group_vars/servers` :

```
/etc/ansible/group_vars/servers
```

Copy

```
ansible_python_interpreter: /usr/bin/python3
```

## Host key verification failed

The first time the control node connects to a managed node over SSH, the remote host key is unknown, and a strict SSH configuration rejects the connection with:

Output

Copy

```
Host key verification failed
```

The simplest fix is to connect to the host once manually so you can review and accept its key, after which Ansible connects without complaint:

Copy

```
$ ssh sammy @ your_server_ip
```

After accepting the key, the host is recorded in `~/.ssh/known_hosts` and subsequent Ansible runs succeed. In disposable lab environments where hosts are frequently rebuilt, some people disable the check by setting `host_key_checking = False` in an `ansible.cfg` file, but leave host key checking enabled for any production work, since it protects against connecting to an impersonated host.

---

# Comparing Rocky Linux 9, AlmaLinux 9, and CentOS Stream ^

Rocky Linux 9 is one of several RHEL-compatible distributions, and the Ansible setup is nearly identical across them, which is useful to know if your environment mixes distributions. The following table summarizes the practical differences for installing Ansible.

| Distribution    | Relationship to RHEL 9                | Ansible install command                    | Notes                                             |
|-----------------|---------------------------------------|--------------------------------------------|---------------------------------------------------|
| Rocky Linux 9   | Downstream rebuild, binary-compatible | <code>sudo dnf install ansible-core</code> | <code>ansible-core</code> 2.14.x from AppStream   |
| AlmaLinux 9     | Downstream rebuild, binary-compatible | <code>sudo dnf install ansible-core</code> | Same AppStream and EPEL repos; commands identical |
| CentOS Stream 9 | Upstream of RHEL 9 (rolling)          | <code>sudo dnf install ansible-core</code> | Tracks slightly ahead of the stable RHEL releases |

In practice, the commands in this guide work without modification on AlmaLinux 9, because it uses the same AppStream and EPEL repositories and the same `ansible-core` package version. CentOS Stream 9 sits upstream of RHEL 9, so it can receive package updates slightly earlier, but the workflow is the same. For enterprise environments that want predictable, long-lived versions, the AppStream `ansible-core` package is the most conservative choice on any of these distributions, while EPEL or pip suits teams that need newer collections or features.

## FAQs ^

### 1. How do I upgrade Ansible on Rocky Linux 9?

How you upgrade depends on how you installed Ansible. If you used the AppStream `ansible-core` package, run `sudo dnf upgrade ansible-core` ; note that AppStream holds `ansible-core` at 2.14 for the RHEL 9 lifecycle, so this only applies bug and security fixes within that series rather than moving you to a newer engine. If you installed the bundled `ansible` package from EPEL, `sudo dnf upgrade ansible` follows EPEL's release. If you installed with pip, upgrade inside your virtual environment with `pip install --upgrade ansible` . To move from the held AppStream version to a newer engine, switch installation methods to EPEL or pip rather than expecting AppStream to advance.

## 2. Why does Ansible report a host as UNREACHABLE?

An `UNREACHABLE` result means Ansible could not establish a working SSH session and run code on the host, as opposed to a task failing once connected. Common causes are an SSH key that is not installed on the managed node, the wrong remote user (set `ansible_user` for the group or host), SSH listening on a non-default port the firewall blocks, or the host simply being down. Run the command again with `-vvv` to see the underlying SSH error, then confirm you can reach the host with a plain `ssh` command using the same user and key.

## 3. How do I install Ansible collections when using `ansible-core` ?

The `ansible-core` package only includes the built-in `ansible.builtin` modules. When a playbook needs a module from a community collection, install that collection with `ansible-galaxy` , for example `ansible-galaxy collection install community.general` . If you installed the full `ansible` package from EPEL or pip instead, the most common collections are already bundled and you usually do not need this step.

## 4. Is Ansible a CI or CD tool?

Ansible is a configuration management and automation tool, not a continuous integration (CI) tool. It describes and enforces the desired state of servers and applications, and it can play a role in the continuous delivery (CD) stage of a pipeline by deploying and configuring infrastructure. It is distinct from CI tools such as Jenkins or GitHub Actions, which build and test code; in many setups, a CI tool triggers an Ansible playbook as the deployment step.

## 5. Can I use Ansible on Windows as a control node?

Native Windows is not supported as an Ansible control node, but running Ansible inside Windows Subsystem for Linux (WSL2) is fully supported and is the recommended

approach for Windows users. The official Ansible documentation lists a WSL distribution as a valid control node platform, so installing Ansible in WSL2 gives you a fully supported Linux control node on a Windows machine. Note that Windows is also fully supported as a *managed node*; the restriction applies only to running the control node natively.

## 6. Do managed nodes need Ansible installed?

Managed nodes do not need Ansible installed. They only require Python 3 and SSH access, because the control node connects over SSH and runs the necessary code remotely. This agentless design is one of Ansible's main advantages: any server with an exposed SSH port and a Python interpreter can be managed without installing extra software on it.

## 7. How do I use a custom inventory file instead of the default `/etc/ansible/hosts` ?

Pass your inventory file to any command with the `-i` flag, for example `ansible all -i inventory.ini -m ping` or `ansible-playbook -i inventory.ini playbook.yml`. To make a custom inventory the default for a project, set the `inventory` directive in an `ansible.cfg` file in the project directory, such as `inventory = ./inventory.ini`. A project-local inventory keeps each project self-contained and is easy to track in version control, which is why it is preferred over editing the system-wide file.

## 8. What is the difference between the `ansible` and `ansible-core` packages?

`ansible-core` is the minimal runtime: the engine plus a small set of built-in modules. The `ansible` package bundles `ansible-core` together with a large, curated set of community collections, so it is much larger but ready to use out of the box. Install `ansible-core` when you want a lean install and will add only the collections you need, and install `ansible` when you want the broad collection set available immediately.

---

## Conclusion



Your Ansible control node is now configured to communicate with the servers you want to control. You verified connectivity with the `ping` module, ran ad-hoc commands, executed a basic playbook, and learned how to handle the most common connection

issues on Rocky Linux 9, including firewalld, SELinux, missing Python interpreters, and SSH host key errors.

You have built a solid foundation for working with your servers through Ansible, so your next step is to learn how to use Playbooks to do the heavy lifting for you. You can learn more in our guide on [Configuration Management 101: Writing Ansible Playbooks](#).

---

## Further reading ^

To continue building your Ansible skills, explore these related DigitalOcean tutorials:

- [Configuration Management 101: Writing Ansible Playbooks](#)
- [How To Execute Ansible Playbooks to Automate Server Setup](#)
- [How to Use Ansible: A Reference Guide](#)
- [How To Use Ansible Vault to Protect Sensitive Playbook Data](#)
- [An Introduction to Configuration Management with Ansible](#)

---

Thanks for learning with the DigitalOcean Community. Check out our offerings for compute, storage, networking, and managed databases.

[Learn more about our products](#) →

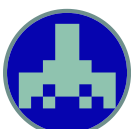
## About the author(s)



**Brian Hogan** Author

[See author profile](#)

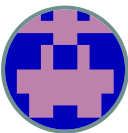
Managed the Write for DOnations program, wrote and edited community articles, and makes things on the Internet. Expertise in DevOps areas including Linux, Ubuntu, Debian, and more.



**Jeanelle Horcasitas** Author  
Technical Writer

[See author profile](#)

Educator and writer committed to empowering our community by providing access to the knowledge and tools for making creative ideas into a reality



**Manikandan Kurup** Editor  
Senior Technical Content Engineer I

[See author profile](#)

With over 6 years of experience in tech publishing, Mani has edited and published more than 75 books covering a wide range of data science topics. Known for his strong attention to detail and technical knowledge, Mani specializes in creating clear, concise, and easy-to-understand content tailored for developers.

**Category:** Tutorial



**Tags:** Ansible Configuration Management  
Rocky Linux 9

Still looking for an answer?

Ask a question

Search for more help

Was this helpful?

Yes

No

Comments(0) Follow-up questions(0)

B I U H<sub>1</sub> H<sub>2</sub> H<sub>3</sub> “”

Leave a comment...

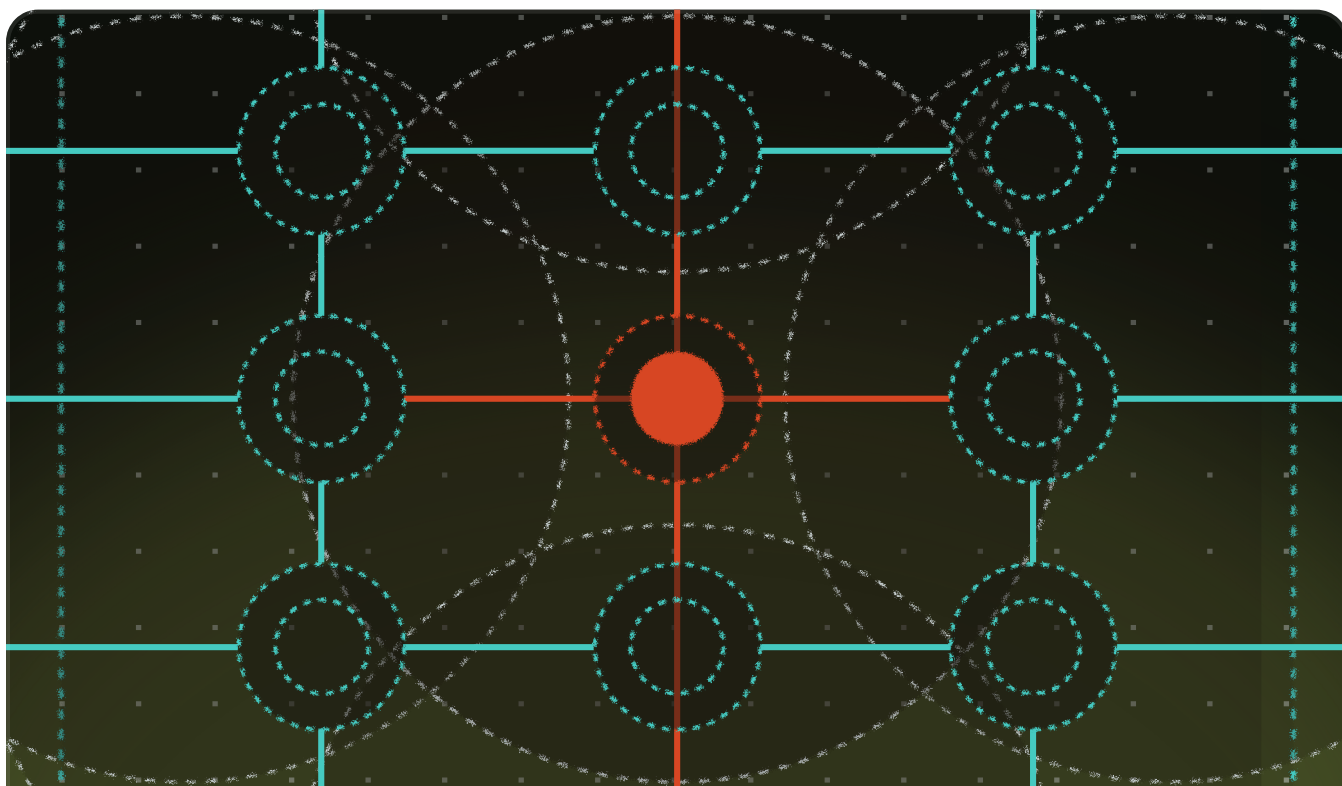
This textbox defaults to using **Markdown** to format your answer.

You can type `!ref` in this text area to quickly search our full set of tutorials, documentation & marketplace offerings and insert the link!

**Sign in/up to comment**



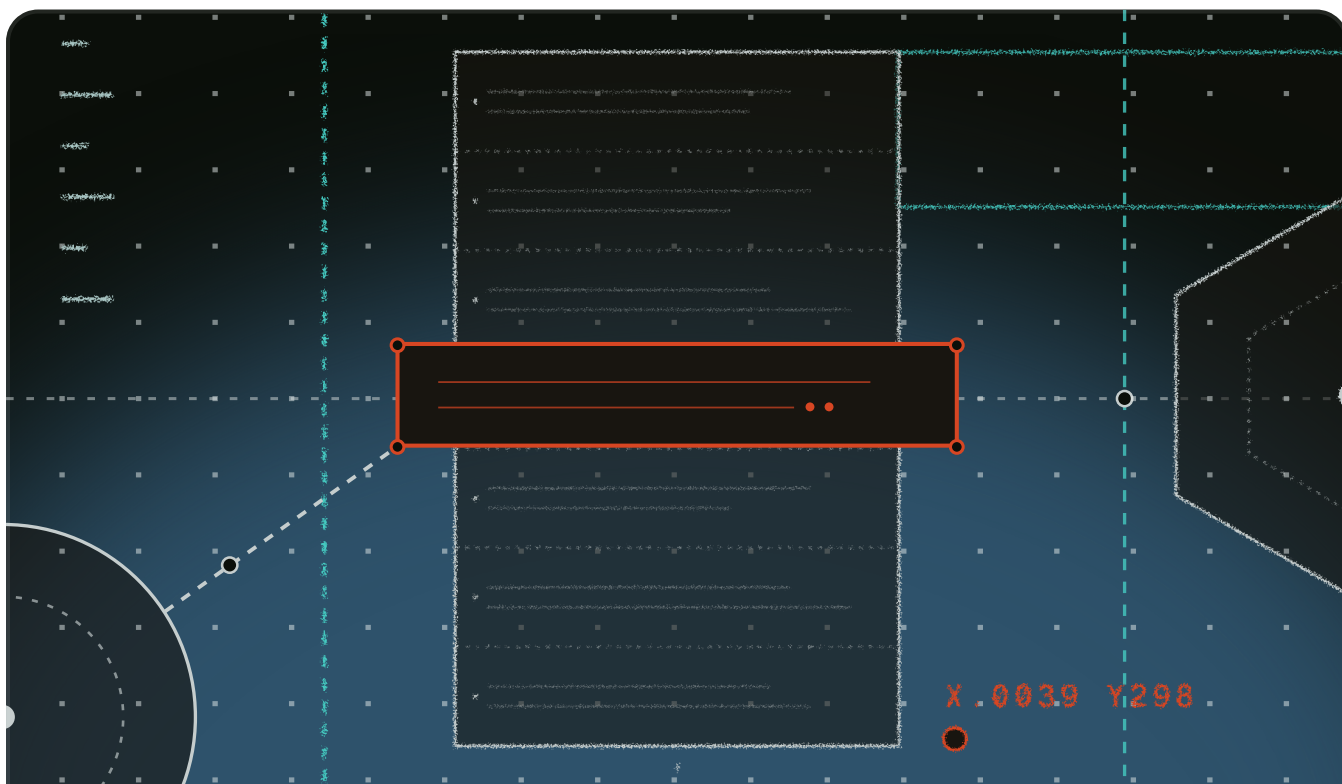
This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.



## Станьте участником сообщества

Получайте деньги за написание технических руководств и выберите благотворительную организацию, связанную с технологиями, чтобы получить аналогичную сумму в качестве пожертвования.

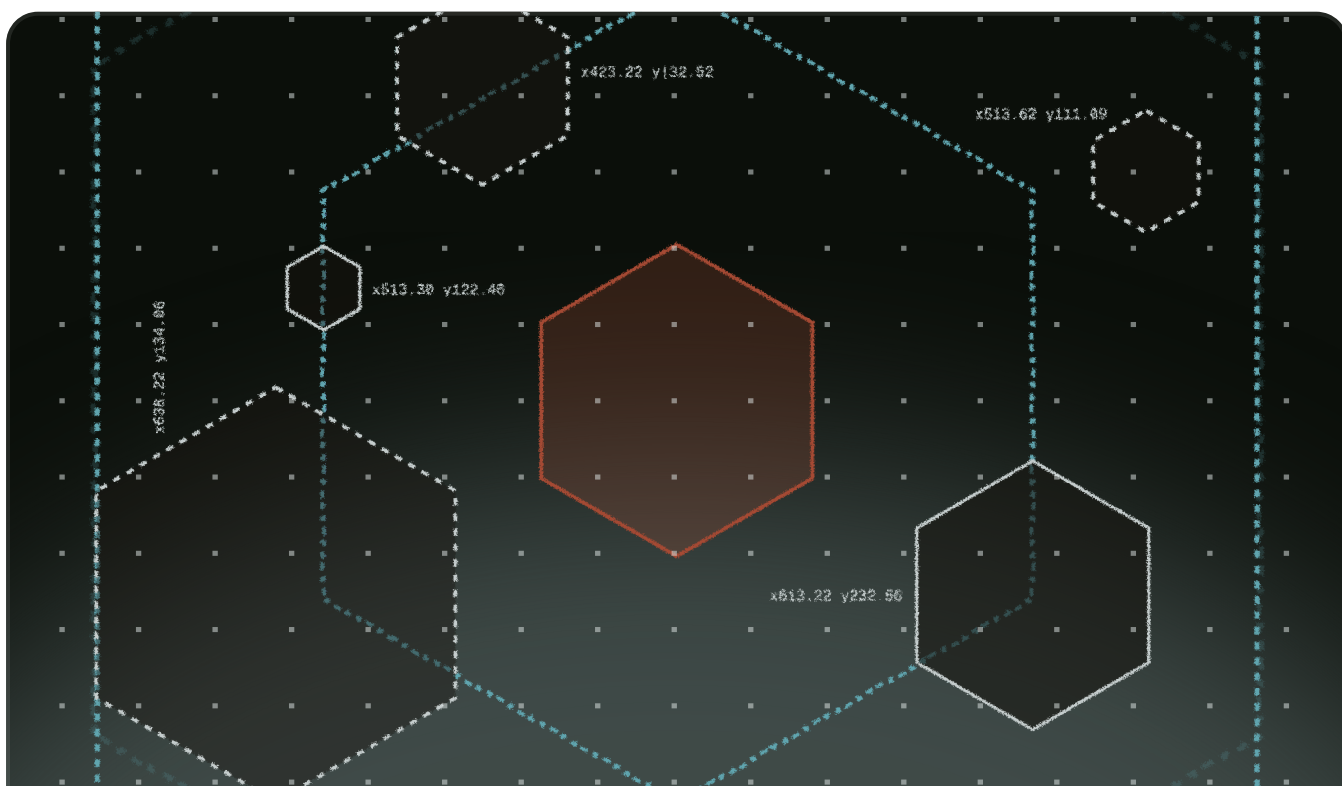
**Регистрация** →



## Документация DigitalOcean

Полная документация по каждому продукту DigitalOcean.

Узнать больше →



## Ресурсы для стартапов и компаний, использующих искусственный интеллект

В The Wave есть все, что вам нужно знать о построении бизнеса: от привлечения финансирования до маркетинга вашего продукта.

Узнать больше →

## Будьте в курсе событий, подпишитесь на нашу рассылку

Электронная почта \*

Отправить

## Облако для разработчиков

Масштабируйтесь по мере роста — независимо от того, используете ли вы одну виртуальную машину или десять тысяч.

Просмотреть все продукты

## Начните создавать уже сегодня

От логического вывода на базе графического процессора и Kubernetes до управляемых баз данных и хранилищ — у нас есть все необходимое для создания,

масштабирования и развертывания интеллектуальных приложений.

Регистрация

Компания



Products



Resources



Solutions



Contact



© 2026 DigitalOcean, LLC. [Sitemap](#). [Cookie Preferences](#)

